Algorithms, 2026 Fall, Homework 6
(Due: Nov 4)

October 26, 2026

You can discuss homeworks with your classmates. But you will have to write down the solution on your own. You need to acknowledge other students who help you in the homework. If you read some other source that is helpful, you should list all of them.

Problem 1 :(Minimum Dominating Set) Given a undirected graph $G(V, E)$, a subset S of vertices is a dominating set if for every $v \in V \setminus S$, there exists a vertex $u \in S$ which is a neighbor of v. The decision version of the minimum dominating set problem asks whether there is a dominating set of size at most k for a given undirected graph $G(V, E)$ and integer k. Prove the problem is NP-Complete.

Problem 2 : Prove the Hamitonian path problem (with specified starting point s and ending point t) in undirected graph is NP-Complete. (A Hamitonian path is a path that visit every node of the graph exactly once). You may assume that the Hamitonian cycle problem in either directed or undirected graphs is NP-Complete.

Problem 3 : Consider the following: Suppose we can solve an NP-Complete problem in quasi-polynomial time (e.g., $O(n^{\log n})$). Show that we can solve all NP problems in this time.

In fact, many TCS researchers believe that an NP-Complete problem requires at least exponential time i.e., $O(2^n)$, and anything substantially smaller (e.g., $2^{\sqrt{n}}$) is unlikely (this is called Exponential Time Hypothesis, which is a stronger conjecture than $P \neq NP$).

Problem 4 : (Max-clique and Densest k-Subgraph) (1) clique in a graph G is a complete subgraph of G. The decision version of the maximum clique problem is as follows: Given a undirected graph G and an positive integer k, is there a clique in G of size at least k. Prove the problem is NP-Complete.

(2) Consider the densest k subgraph problem. We are given an undirected graph G and positive integers k and Z. The densest k-subgraph problem asks whether there is a subgraph of G with k vertices such that the total number of edges in the subgraph is at least Z. Prove this problem is NP-Complete.

Problem 6 : In formal complexity theory, we should understand a decision problem L (formally it is called a language) as an (infinite) collection of yes instances (the instances for which we should output yet). Hence, we can formally define *the complement* $\bar{L}$ of a language L, which consists of all problem instances not in L (or all no

instances). Define the class co-NP to be all languages that are complement of some NP languages. For example, the problem of deciding whether the graph does not contain a vertex cover of size at most k is a co-NP problem.

It may appears that NP and co-NP are equivalent to each other since for any input instance we only need to change a yes answer to a no answer. But (people believe) there is fundamental difference between these two classes. For an NP problem L and a YES instance (a string $X \in L$), there exists a certificate (or "solution", the input t in $B(s,t)$ mentioned in the class) that we can verify in poly-time. For example, we verify whether an assignment satisfies a 3CNF. However, for a co-NP problem $\bar{L}$ and a string $X \in \bar{L}$), it is unclear such a certificate exists. For example, how can you quickly verify whether the graph does not contain a vertex cover of size at most k (without trying all possibilities). Make sure you understand the difference.

1. Prove P$\subseteq$ NP $\cap$ co-NP.

2. Define PRIME as the problem of deciding whether a positive number is a prime. The problem is known in P (through a celebrated algorithm called AKS primality test). Hence PRIME is in NP $\cap$ co-NP. (note that in this problem, poly-time means poly in the length of the input, encoded in binary bits).

   Now consider the FACTOR problem, which generalize the above problem. The input is of the form (x, r) and the goal is to decide whether the given number x has a prime factor that is smaller than r. Show FACTOR is in NP $\cap$ co-NP.

Notes: All reductions you learn in our class are not very complicated. If you are interested, you can take a look at the following more difficult examples.

(1) The paper shows that Tetris is NP-Hard

`http://publications.csail.mit.edu/publications/pubs/pdf/MIT-LCS-TR-865.pdf`

(2) This paper shows that the minimum triangulation problem is NP-hard. This was an open problem for a long time and the problem was solved with computer assistance (some gadgets are designed with the help of computer).

`https://arxiv.org/pdf/cs/0601002.pdf`